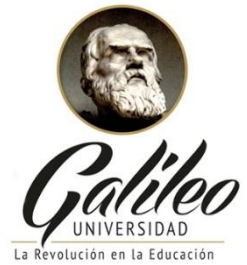


Universidad Galileo	Fecha: 23/09/2026
Facultad: FISSICC	Alumno: Angel Camargo
Curso: Microprocesadores	Carné: 24003664
Sección: A	Hora de Laboratorio: 1:00 pm
Auxiliar: Brandon Eduardo	Día de Laboratorio: Lunes



Laboratorio #6 Puertos Paralelos (GPIO)

Objetivos:

Adquirir los conocimientos para poder configurar y controlar los puertos de propósito general GPIO del microcontrolador correspondiente utilizando como lenguaje C y CMSIS. Comprender el funcionamiento de los registros para poder habilitar los puertos, configurar cada uno de los pines correspondientes como entradas o salidas según corresponda y leer o modificar estados lógicos.

Resumen:

Durante esta práctica trabajamos con puertos de propósito general, también llamados como GPIO para el microcontrolador correspondiente. Primero se decidió cuales de los pines íbamos a utilizar como salida y cuales pines iban a ser entrada, las salidas iban conectadas a los leds. Primero que nada, se tuvo que habilitar el reloj correspondiente a cada GPIO, procedente a esto configuramos las resistencias como pull-down o pull-up según como quisiéramos nuestra configuración, en este caso pull-down, cuando se presionara el led este iba a encender y cuando no se presionara se mantendría apagado.

Todo esto lo logramos haciendo uso de mascarar para poder modificar los bits correspondientes según como quisieras nuestra configuración.

Teoría:

Puertos GPIO

Se les conoce como puertos GPIO a aquellos pines de propósito general que se pueden configurar para funcionar como entradas o salidas digitales. Los microcontroladores de las familias STM32F303 y STM32G431 pueden tener uno o más puertos GPIO identificador por medio de letras, como es el caso del microcontrolador STM32G431KBT6, que viene con GPIOA, GPIOB, GPIOC etc.

Registro MODER

Este registro nos permite establecer el modo de funcionamiento de cada uno de los pines del GPIO correspondiente, cada pin utiliza dos bits dentro de estos registros los cuales son: 00 para entrada, 01 para salida, 10 para función alternativa y 11 para modo analógico.

Registro IDR

Este registro, nos permite poder leer el estado lógico actuales de los pines de un puerto. Cuando uno de los pines ha sido configurado como entrada, su valor puede consultarse por medio de este registro para poder determinar el estado único del pin, utilizamos una mascara y una operación AND.

Registro ODR

Este es otro tipo de registro, el cual básicamente nos permite poder modificar el estado lógico de los pines configurados como salida. Para colocar un pin en estado 1 podemos utilizar la operación OR junto con una máscara, mientras que para colocarla en 0 utilizamos un AND y una máscara, de esta forma modificamos correctamente el estado del pin que queramos modificar.

Registro PUPDR

Los pines que fueron configurados como entradas pueden utilizar resistencias internas, las cuales pueden ser pull- up o pull down mediante el registro PUPDR, cada secuencia de bits tienen diferentes configuraciones, las cuales son: 00 sin pull-up ni pull-down , el 01 es un pull-up, el 10 es un pull-down y el 11 es reservado.

Datos Prácticos:

Activar reloj

```
int main(void){  
    RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;  
    RCC->AHB2ENR |= RCC_AHB2ENR_GPIOBEN;
```

Configurar salidas

```
GPIOA->MODER &= ~(3U << (2 * 7));
GPIOA->MODER |= (1U << (2 * 7));

GPIOA->MODER &= ~(3U << (2 * 4));
GPIOA->MODER |= (1U << (2 * 4));

GPIOA->MODER &= ~(3U << (2 * 0));
GPIOA->MODER |= (1U << (2 * 0));

GPIOA->MODER &= ~(3U << (2 * 2));
GPIOA->MODER |= (1U << (2 * 2));

GPIOA->MODER &= ~(3U << (2 * 6));
GPIOA->MODER |= (1U << (2 * 6));
```

Configurar entradas y resistencias en pull-down

```
GPIOB->MODER &= ~(3U << (2 * 0));
GPIOB->PUPDR &= ~(3U << (2 * 0));
GPIOB->PUPDR |= (2U << (2 * 0));

GPIOB->MODER &= ~(3U << (2 * 7));
GPIOB->PUPDR &= ~(3U << (2 * 7));
GPIOB->PUPDR |= (2U << (2 * 7));

GPIOB->MODER &= ~(3U << (2 * 6));
GPIOB->PUPDR &= ~(3U << (2 * 6));
GPIOB->PUPDR |= (2U << (2 * 6));

GPIOB->MODER &= ~(3U << (2 * 5));
GPIOB->PUPDR &= ~(3U << (2 * 5));
GPIOB->PUPDR |= (2U << (2 * 5));

GPIOB->MODER &= ~(3U << (2 * 4));
GPIOB->PUPDR &= ~(3U << (2 * 4));
GPIOB->PUPDR |= (2U << (2 * 4));
```

Alternar lógica para que los leds se pagan o enciendan si se presiona el botón o no

```
while (1){  
    if (GPIOB->IDR & (1U << 0)){  
        GPIOA->ODR |= (1U << 2);  
    }  
    else {  
        GPIOA->ODR &= ~(1U << 2);  
    }  
  
    if (GPIOB->IDR & (1U << 7)) {  
        GPIOA->ODR |= (1U << 7);  
    }  
    else{  
        GPIOA->ODR &= ~(1U << 7);  
    }  
  
    if (GPIOB->IDR & (1U << 6)){  
        GPIOA->ODR |= (1U << 4);  
    }  
    else{  
        GPIOA->ODR &= ~(1U << 4);  
    }  
    if (GPIOB->IDR & (1U << 5)){  
        GPIOA->ODR |= (1U << 0);  
    }  
    else {  
        GPIOA->ODR &= ~(1U << 0);  
    }  
  
    if (GPIOB->IDR & (1U << 4)) {  
        GPIOA->ODR |= (1U << 6);  
    }  
    else{  
        GPIOA->ODR &= ~(1U << 6);  
    }  
}
```

Videos de demostración del laboratorio:

<https://drive.google.com/drive/folders/1SvB5BmxGwrjV7hhDy8g-fWE0xSVkKDP?usp=sharing>

Datos Teóricos

Utilizando una resistencia de 220Ω

$$I = \frac{V_{GPIO} - V_{LED}}{R}$$

$$I = \frac{3.3 - 1.8}{220}$$

$$I = 6.8mA$$

Esta corriente es suficiente para poder encender el led y como podemos ver se mantiene por debajo del límite de 15mA el cual es indicado en la guía.

Conclusiones:

- La configuración de los puertos GPIO requiere poder habilitar primeramente la señal del reloj del periférico que vamos a usar por medio del módulo RCC, y después configurar cada uno de los pines según sea necesario
- Cada uno de los registros, MODER, IDR, ODR, PUPDR nos permite poder modificar y controlar directamente el comportamiento de cada uno de los pines del microcontrolador según nuestra necesidad.
- Con el uso de mascarar y operaciones de bit a bit podemos modificar los bits necesarios dentro de un registro sin alterar la configuración de los demás pines y así poder modificar la lógica de cada uno de estos
- Mediante la lectura de los registros IDR y la escritura de los registros ODR podemos implementar una comunicación directa entre dispositivos de entrada y dispositivos de salida.

Bibliografía:

STMicroelectronics, "STM32G4 series - Documentation," *STMicroelectronics*. [Online]. Available: <https://www.st.com/en/microcontrollers-microprocessors/stm32g4-series/documentation.html>. [Accessed: Oct. 4, 2026]

Arm Ltd., “CMSIS,” *Arm Developer Documentation*, ver. 6. [Online]. Available: <https://support.arm.com/documentation/109350/v6/CMSIS>. [Accessed: Oct. 4, 2026]

STMicroelectronics, *STM32G4 Nucleo-32 Board (MB1430) User Manual*, UM2397. [Online]. Available: https://www.st.com/resource/en/user_manual/um2397-stm32g4-nucleo32-board-mb1430-stmicroelectronics.pdf. [Accessed: Oct. 4, 2026]

STMicroelectronics, *STM32G4 Series Advanced Arm-Based 32-Bit MCUs Reference Manual*, RM0440. [Online]. Available: https://www.st.com/resource/en/reference_manual/dm00355726-stm32g4-series-reference-manual-stmicroelectronics.pdf. [Accessed: Oct. 4, 2026]